

Giới thiệu List trong Java



NỘI DUNG

1. Vị trí của List trong Collection Framework

2. List là gì?

3. ArrayList vs LinkedList

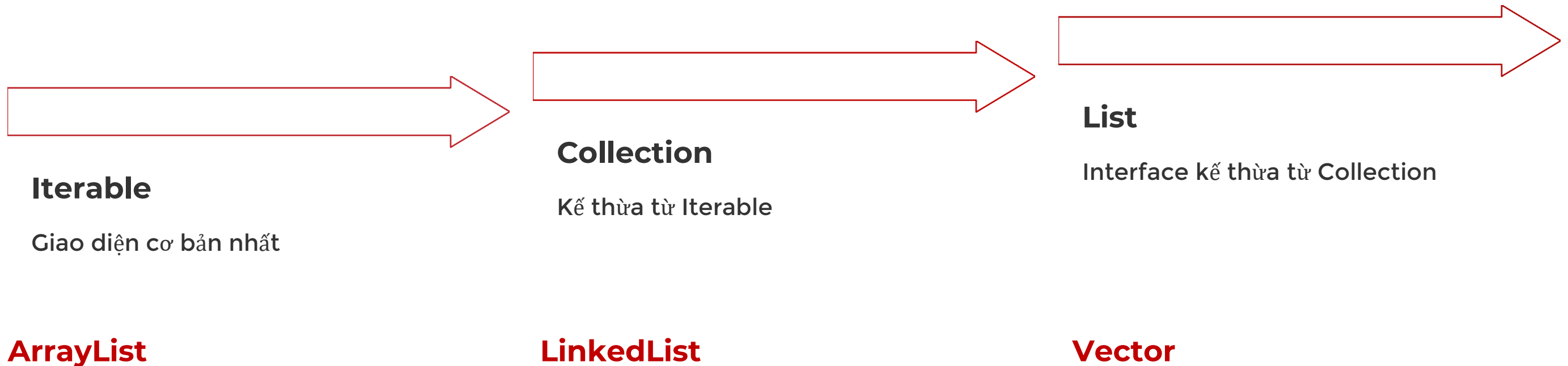
4. Cách duyệt List

5. Một số lưu ý

6. Kết luận

Ôn lại Collection

1. List nằm ở đâu trong Collection Framework?



List là một interface, kế thừa từ Collection. Các lớp phổ biến nhất của List là ArrayList và LinkedList.

Khái niệm cơ bản

2. List là gì?

1

Danh sách có thứ tự

Các phần tử được sắp xếp theo vị trí



Cho phép phần tử trùng lặp

Có thể chứa các giá trị giống nhau



Truy cập qua index

Truy xuất phần tử theo vị trí

List giống như một mảng nâng cao: có thứ tự, có vị trí và có thể chứa các phần tử trùng nhau.

Vì sao dùng List?

So sánh List & Array

Array	List
Kích thước cố định	Kích thước linh hoạt
Ít hàm hỗ trợ	Nhiều hàm tiện ích
Xử lý thủ công	Xử lý nhanh, gọn

📌 **Kết luận:** Khi số lượng phần tử thay đổi liên tục, List là lựa chọn tối ưu hơn mảng.

Các implementation phổ biến

3. ArrayList vs LinkedList

ArrayList

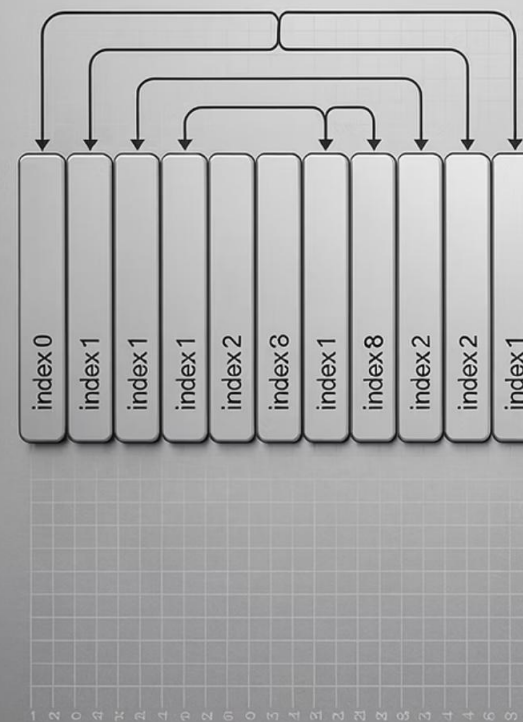
- Truy cập nhanh
- Chậm khi thêm/xóa giữa

LinkedList

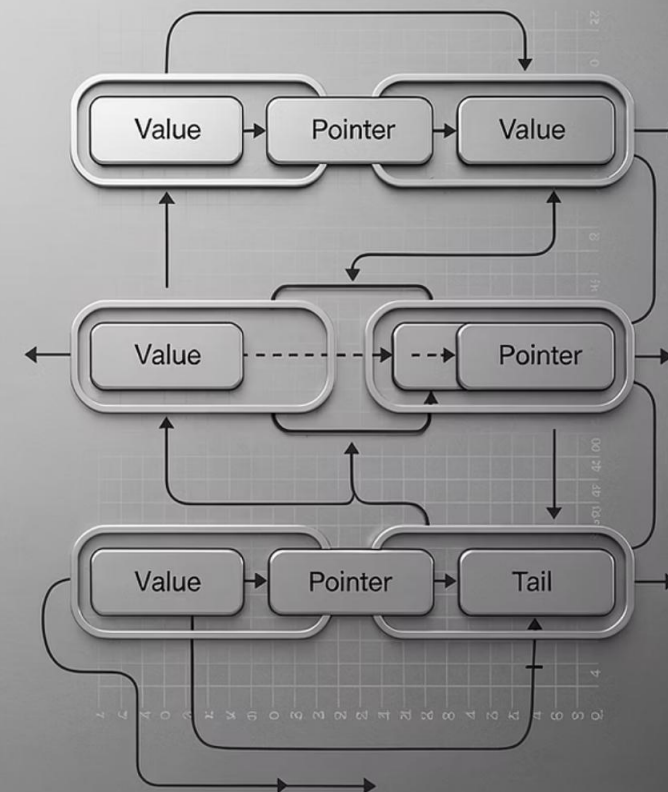
- Thêm/xóa nhanh
- Truy cập chậm hơn

Việc chọn ArrayList hay LinkedList phụ thuộc vào nhu cầu sử dụng: truy cập hay chỉnh sửa dữ liệu.

ArrayList



LinkedList



```
Fredaw  Worlrdny  Striesblp
...  ts/ormend  x  Java anik  x
1  import java.util.*;
2
3  list tjava util<*>;
4
5  list(String) names = new ArrayList<>();
6  {
7      class Cclassleg: "(import"=java util=)"; []
8      class ([dens:"loderelutl"]
9          class:"dvoiridei:Poy-"edinellong";
10         list:"(Ionsslioc:"deduright"tavid:oPratudtnxssPiles°);
11         tlss:"(oanerldnen-thetr"editaLJdiætong"";
12         list("Class:"inEilUangd")
13         teares:"le";
14         slhers:"le");
15     };
16 }
17
18 java seed []
19
20 / new audise;fay--
21
22
```

Cách tạo List

Khai báo List

```
List<String> names = new ArrayList<>();
```

Chúng ta thường khai báo theo interface List để dễ dàng thay đổi implementation khi cần.

Nhóm phương thức thêm phần tử

add()

```
list.add("Java");  
list.add(1, "Python");
```

- `add(E e)`
- `add(int index, E e)`

Phương thức add giúp thêm phần tử vào cuối hoặc vào vị trí bất kỳ trong List.

Lấy phần tử

get()

```
String value = list.get(0);
```



Truy cập theo index

Sử dụng chỉ số để lấy phần tử



Index bắt đầu từ 0

Phần tử đầu tiên có vị trí 0

Truy cập phần tử trong List rất giống mảng, nhưng an toàn và linh hoạt hơn.

Cập nhật phần tử

set()

```
list.set(1, "C++");
```

→ **Thay thế giá trị tại index**

Cập nhật phần tử ở vị trí chỉ định

→ **Không thay đổi kích thước List**

Chỉ thay đổi giá trị, không thêm mới

set dùng để cập nhật phần tử, không phải thêm mới.

Xóa phần tử

remove()

```
list.remove(0);  
list.remove("Java");
```

Hai cách xóa:

- **Xóa theo index**
- **Xóa theo giá trị**

remove là phương thức rất hay dùng khi cần chỉnh sửa danh sách dữ liệu.

Các phương thức thường dùng khác

Các method quan trọng

1

size()

Trả về số lượng phần tử trong List

2

contains()

Kiểm tra xem List có chứa phần tử hay không

3

isEmpty()

Kiểm tra List có rỗng hay không

4

clear()

Xóa tất cả phần tử trong List

Đây là những phương thức giúp kiểm tra và quản lý List một cách nhanh chóng.

Duyệt List

4. Cách duyệt List

```
for(String s : list) {  
    System.out.println(s);  
}
```

Cách duyệt phổ biến nhất là for-each, vừa gọn vừa dễ đọc.

Lưu ý khi làm việc với List

5. Một số lưu ý

**Tránh
IndexOutOfBoundsException**

Luôn kiểm tra index trước khi truy cập

**Không xóa phần tử khi
đang for-each**

Sẽ gây ra

ConcurrentModificationException

Nên dùng Generic

Đảm bảo type-safe và tránh lỗi
runtime

📌 **Quan trọng:** Hiểu rõ các lỗi thường gặp sẽ giúp bạn code List an toàn hơn.

Tổng kết

6. Kết luận



01

**List là Collection dùng
nhiều nhất**

02

**Có nhiều phương thức
mạnh mẽ**

03

**Là nền tảng cho các bài
nâng cao**